



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A05 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 24 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN4

ENUNCIADO.....5

 Probad todo el código del Apartado 55

 Probad los Scripts del Apartado 57

 Caso práctico 510

 Ponte a prueba 512

 Claves de resolución:.....12

DESARROLLO DE LA PRACTICA13

 Descripción13

 Contenido del archivo13

 Código del Apartado 5 de teoría13

 Probar los Scripts del Apartado 515

 Versión 1: recorrido con forEach anidado y Object.values() para mostrar estructuras de objetos.
 16

 Versión 2: presentación alternativa con formato un poco mas limpio y ordenado.17

 Versión 3: mejora visual usando funciones como reduce() y plantillas literales para generar
 salidas más legibles y estructuradas.17

 Caso Práctico 5 – Trabajo con arrays de objetos.....17

 1. Operaciones sobre array de coches17

 Ponte a Prueba 5 - Cálculo de medias (variante funcional)19

 Cómo visualizarlo20

 Requisitos:20

 • Editor de código compatible: en mi caso, Visual Studio Code.20

 • Navegador web: para ejecutar scripts con console.log() o integrarlos en una interfaz HTML
 (que no es mi caso).....20

 • Node.js (opcional): para ejecutar los scripts directamente desde la terminal del equipo o del
 VS Code.....20

 Pasos:.....20

 • Guardar todos los archivos .js en la carpeta raíz del proyecto.20

 • Abrir el o los archivos en el editor de código.20

 • Ejecutar el script en consola con Node.js o abrirlo directamente en la consola del navegador
 (F12).....20

 • Ver los resultados por consola y comprobar que todas las funciones devuelven los valores
 esperados.20

Objetivos de la práctica20

 • Aplicar programación funcional para recorrer, transformar y filtrar arrays de objetos.20

- Utilizar métodos como filter(), find(), map(), reduce() y forEach() de forma eficiente.20
- Crear funciones reutilizables con parámetros rest.20
- Intentar mejorar la presentación de datos en consola.20
- Documentar correctamente cada ejercicio con autoría y capturas.....20
- Presentar el proyecto en formato PDF junto con los archivos fuente y capturas de pantalla.
20

CONCLUSIÓN21

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 5” deberemos probar, en uno o varios fuentes (recomendable varios siempre que se pueda) el código que aparezca en el Apartado 5 de la teoría de JavaScript, así como su ejecución y autoría.

En el ejercicio “Probad los Scripts del Apartado 5” deberemos ejecutar los scripts adjuntos en la propia tarea y explicar con comentarios en el mismo script su funcionamiento. Muy importante, en el tercer fuente habrá que intentar mostrar el siguiente resultado por consola adjunto en el ejercicio. Además, luego se deberá mostrar una segunda versión del resultado anterior.

En el “Caso Práctico 5”, a través de un array dado, deberemos realizar las siguientes operaciones mediante la Programación Funcional. El resultado deberá ser igual o muy similar a la foto adjunta en la explicación del ejercicio.

En el “Ponte a Prueba 5” supuestamente habremos estudiado el apartado de Funciones, pues resolveremos un caso práctico utilizando código funcional en vez de bucles convencionales. La función será calcular la media aritmética de valores pasados como parámetros.

ENUNCIADO

Probad todo el código del Apartado 5

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWEC_U02_A05_TEO_1.js

```
JS DWEC_U02_A05_TEO_1.js X
JS DWEC_U02_A05_TEO_1.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/10/2025
3
4
5  // 5.1. Fundamentos: la función "filter"
6  let datos = [14, 22, 16, 3, 35];
7
8  // Obtener los elementos menores que 15
9  let resultado = [];
10 for(let dato of datos) {
11     if(dato < 15) {
12         resultado.push(dato);
13     }
14 }
15
16 console.log(resultado); // [14, 3]s
17
18
19 let resultado2 = datos.filter(function (dato) {
20     return dato < 15;
21 });
22
23 console.log(resultado); // [14, 3]
24
25
26 // Función que comprueba si un dato es menor que 15
27 // Se ha utilizado una función flecha en este caso
28 // Como solo tiene un parametro, se pueden omitir los paréntesis
29 // Como solo tiene una línea en el cuerpo, se pueden omitir las llaves y el 'return'
30 // Devuelve true si se cumple la condición y false en caso contrario
31 let menorQue15 = dato => dato < 15;
32
33 console.log(resultado); // [14, 3]
```

Captura de pantalla – DWEC_U02_A05_TEO_2.js

```
JS DWEC_U02_A05_TEO_2.js X
JS DWEC_U02_A05_TEO_2.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/10/2025
3
4
5  // 5.2. "Find"
6  let usuarios = [
7    {nombre: "David", apellido: "Pérez", nif: 1},
8    {nombre: "Juan", apellido: "Martínez", nif: 2},
9    {nombre: "José", apellido: "Fernández", nif: 3}
10 ]
11
12 let usuarioConNif2 = usuarios.find(item => item.nif == 2);
13 console.log(usuarioConNif2); // {nombre: "Juan", apellido: "Martínez", nif: 2},
```

Captura de pantalla – DWEC_U02_A05_TEO_3.js

```
JS DWEC_U02_A05_TEO_3.js X
JS DWEC_U02_A05_TEO_3.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/10/2025
3
4
5  // 5.3. "ForEach"
6  let datos = ["a", "b", "c", "d"];
7  datos.forEach(alert);
8
9
10 datos.forEach(function(item, indice, array) {
11   console.log(`Procesando el elemento ${item}, en la posición ${indice}. El array es ${array}`);
12 });
13
14 // Resultado:
15 // Procesando el elemento a, en la posición 0. El array es a,b,c,d.
16 // Procesando el elemento b, en la posición 1. El array es a,b,c,d.
17 // Procesando el elemento c, en la posición 2. El array es a,b,c,d.
18 // Procesando el elemento d, en la posición 3. El array es a,b,c,d.
```

Captura de pantalla – DWEC_U02_A05_TEO_4.js

```
JS DWEC_U02_A05_TEO_4.js X
JS DWEC_U02_A05_TEO_4.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/10/2025
3
4
5  // 5.4. "Map"
6  let numeros = [1, 3, 5, 7];
7
8  let dobleNumeros = numeros.map(function(numero) {
9    return numero * 2;
10 });
11
12 // No se modifica el array original
13 console.log(numeros); // [1, 3, 5, 7]
14 console.log(dobleNumeros); // [2, 6, 10, 14]
```

Captura de pantalla – DWEC_U02_A05_TEO_5.js

```
JS DWEC_U02_A05_TEO_5.js X
JS DWEC_U02_A05_TEO_5.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 23/10/2025
3
4
5  // 5.5. "Reduce"
6  let numeros = [2, 5, 4];
7
8  let total = numeros.reduce(
9    // Primer parámetro: función callback
10   function(acc, numeros) {
11     return acc + numeros;
12   },
13   // Segundo parámetro: valor inicial
14   0);
15
16 console.log(total); // 11
```

Probad los Scripts del Apartado 5

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

En el tercer script, intenta mostrar por consola el siguiente resultado (versión 1):

```
{matricula: 'ABC1', color: 'rojo'}
{matricula: 'ABC4', color: 'rojo'}
{matricula: 'ABC2', color: 'verde'}
{matricula: 'ABC3', color: 'azul'}
```

Para ello deberás utilizar uno o varios `forEach` anidados, revisa también el método **`Object.values()`**

Ahora intenta mostrar por consola este otro resultado (versión 2):

```
ABC1
rojo
ABC4
rojo
ABC2
```

verde
ABC3
azul

Por último, intenta mejorar la presentación de las dos versiones anteriores, como creas conveniente, puedes utilizar otras funciones vistas en este apartado, como por ejemplo la función **reduce**, ... (versión 3):

Ejemplo de implementación

Captura de pantalla – DWEC_U02_A05_01.js

```
JS DWEC_U02_A05_01.js X
C:\Users\mario\Desktop\U02_A05 Casos prácticos, Ponte a prueba, etc\Scripts Apartado
5\DWEC_U02_A05_01.js
2 // Fecha: 23/10/025
3
4 let usuarios = [
5   {nombre: "David", apellido: "Pérez", nif: 1},
6   {nombre: "Juan", apellido: "Martínez", nif: 2},
7   {nombre: "José", apellido: "Fernández", nif: 3}
8 ]
9
10 let usuariosMod = usuarios.map(usuario => {
11   // Se modifica el objeto original
12   usuario.nombreCompleto = `${usuario.nombre} ${usuario.apellido}`;
13   // Se devuelve una referencia al mismo objeto original
14   return usuario;
15 })
16
17 console.log(usuariosMod[0].nombreCompleto); // David Pérez
18 console.log(usuariosMod[1].nombreCompleto); // Juan Martínez
19 console.log(usuariosMod[2].nombreCompleto); // José Fernández
20
21 // Comprobamos que el array original apunta a los mismos objetos
22 console.log(usuarios[0].nombreCompleto); // David Pérez
23 console.log(usuarios[1].nombreCompleto); // Juan Martínez
24 console.log(usuarios[2].nombreCompleto); // José Fernández
25
26 // Comprobamos que se apunta al mismo objeto:
27 // Cambiamos el valor a través de 'usuariosMod'
28 usuariosMod[0].nombreCompleto = "aa";
29
30 // Comprobamos el valor a través de 'usuarios'
31 console.log(usuarios[0].nombreCompleto); // aa
```


Captura de pantalla – DWECS_U02_A05_02.js

```
JS DWECS_U02_A05_02.js X
Scripts Apartado 5 > JS DWECS_U02_A05_02.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 23/10/2025
3
4 let usuarios = [
5   {nombre: "David", apellido: "Pérez", nif: 1},
6   {nombre: "Juan", apellido: "Martínez", nif: 2},
7   {nombre: "José", apellido: "Fernández", nif: 3}
8 ]
9
10 let usuariosMod = usuarios.map(usuario => {
11   // Se crea un nuevo objeto vacío
12   let nuevoUsuario = {};
13   // Se copia el objeto original al nuevo objeto
14   Object.assign(nuevoUsuario, usuario);
15   // Se modifica el nuevo objeto
16   nuevoUsuario.nombreCompleto = `${nuevoUsuario.nombre} ${nuevoUsuario.apellido}`;
17   // Se devuelve el objeto nuevo
18   return nuevoUsuario;
19 })
20
21 console.log(usuariosMod[0].nombreCompleto); // David Pérez
22 console.log(usuariosMod[1].nombreCompleto); // Juan Martínez
23 console.log(usuariosMod[2].nombreCompleto); // José Fernández
24
25 // Comprobamos que el array original no se ha modificado
26 console.log(usuarios[0].nombreCompleto); // undefined
27 console.log(usuarios[1].nombreCompleto); // undefined
28 console.log(usuarios[2].nombreCompleto); // undefined
29
30 // Comprobamos que se apunta a objetos distintos:
31 // Cambiamos el valor a través de 'usuariosMod'
32 usuariosMod[0].nombreCompleto = "aa";
33 usuariosMod[0].nombre = "Distinto";
34
35 // Comprobamos el valor a través de 'usuarios'
36 console.log(usuarios[0].nombreCompleto); // undefined
37 console.log(usuarios[0].nombre); // David
```

Captura de pantalla – DWECS_U02_A05_03.js

```
JS DWECS_U02_A05_03.js X
Scripts Apartado 5 > JS DWECS_U02_A05_03.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 23/10/2025
3
4 // Listado de coches
5 let coches = [
6   {matricula: "ABC1", color: "rojo"},
7   {matricula: "ABC2", color: "verde"},
8   {matricula: "ABC3", color: "azul"},
9   {matricula: "ABC4", color: "rojo"}
10 ]
11
12
13 let clasificacion = coches.reduce(function(acc, coche) {
14   // Hay que comprobar si existe una entrada en el objeto para el color del coche actual
15   // El operador '||' ('or' lógico) se comporta como un condicional:
16   // si el primer operando está definido, se devuelve; si no, se devuelve el segundo
17   // En este caso, si existe 'acc[coche.color]', se utiliza;
18   // si no, se crea un objeto nuevo con las propiedades 'cuenta' y 'listado' vacías
19   acc[coche.color] = acc[coche.color] || {cuenta: 0, listado: []};
20   // Se incrementa la cuenta de coches de ese color
21   acc[coche.color].cuenta++;
22   // Se añade el coche al listado del color correspondiente
23   acc[coche.color].listado.push(coche);
24   return acc;
25 }, {}); // El valor inicial es un objeto vacío
26
27 console.log(clasificacion);
28 // {
29 //   rojo: { cuenta: 2, listado: [ [Object], [Object] ] },
30 //   verde: { cuenta: 1, listado: [ [Object] ] },
31 //   azul: { cuenta: 1, listado: [ [Object] ] }
32 // }
33
34 // Cuántos coches hay de color rojo
35 console.log(clasificacion.rojo.cuenta); // 2
36 // Cuántos coches hay de color verde
37 console.log(clasificacion.verde.cuenta); // 1
```

Caso práctico 5

Trabajo con *arrays* de objetos

Dado el siguiente *array* de objetos, realiza las siguientes operaciones mediante programación funcional:

```
let coches = [  
  {id: 1, marca: "Seat", modelo: "Ibiza", precio: 10000},  
  {id: 2, marca: "Seat", modelo: "Ateca", precio: 18000},  
  {id: 3, marca: "Volkswagen", modelo: "Golf", precio:  
    21000},  
  {id: 4, marca: "Kia", modelo: "Niro", precio: 30000},  
];
```

- Obtén los coches de la marca "Seat".
- Obtén los coches cuyo precio sea menor que 25 000.
- Encuentra el primer coche cuyo precio sea mayor que 20 000.
- Transforma el *array* en otro con los mismos campos más uno adicional, denominado **premium**. Si el precio es mayor de 20 000, **premium=true**; en caso contrario, **premium=false**.
- Calcula el precio medio de los coches de la marca "Seat".

Intentad mostrar por consola el siguiente resultado:

Ejemplo de implementación:

```
JS CasoPractico5.js X  
Caso Practico > JS CasoPractico5.js > ...  
1 // Autor: Mario Segura Abad  
2 // Fecha: 24/10/2025  
3  
4 let coches = [  
5   {id: 1, marca: "Seat", modelo: "Ibiza", precio: 10000},  
6   {id: 2, marca: "Seat", modelo: "Ateca", precio: 18000},  
7   {id: 3, marca: "Volkswagen", modelo: "Golf", precio: 21000},  
8   {id: 4, marca: "Kia", modelo: "Niro", precio: 30000}  
9 ];  
10  
11 // =====  
12 // 1. Coches de la marca "Seat"  
13 // =====  
14 let cochesMarcaSeat = coches.filter(coches => coches.marca === "Seat");  
15 console.log("2DAW2\t 24/10/2025\n");  
16 console.log(cochesMarcaSeat);  
17 console.log(); // salto de línea entre coches  
18  
19 // =====  
20 // 2. Coches con precio menor que 25.000  
21 // =====  
22 let cochesMenoresQue25000 = coches.filter(coches => coches.precio < 25000);  
23 console.log(cochesMenoresQue25000);  
24 console.log();  
25  
26 // =====  
27 // 3. Primer coche con preceio mayor que 20.000  
28 // =====  
29 let cocheMayorQue20000 = coches.find(coches => coches.precio > 20000);  
30 console.log(cocheMayorQue20000);  
31 console.log();  
32  
33 // =====  
34 // 4. Array con campo adicional "premium"  
35 // =====  
36 let cochesPremium = coches.map(coches => ({  
37   ...coches,
```

2DAW2 17/10/2025

(2) [{...}, {...}]

(3) [{...}, {...}, {...}]

{id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000}

(4) [{...}, {...}, {...}, {...}]

{total: 28000, cuenta: 2}

2DAW2 17/10/2025

Coches Seat:

(2) [{...}, {...}]

{id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000}

{id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000}

Coches Menores que 25000:

(3) [{...}, {...}, {...}]

{id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000}

{id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000}

{id: 1, marca: 'Volkswagen', modelo: 'Golf', precio: 21000}

1º Coche Mayor que 20000:

{id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000}

3

Volkswagen

Golf

21000

Coches con Premium:

(4) [{...}, {...}, {...}, {...}]

{id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000,
premium: false}

{id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000,
premium: false}

{id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000,
premium: true}

{id: 4, marca: 'Kia', modelo: 'Niro', precio: 30000,
premium: true}

Total y Cuenta Seat

[object Object] , precio medio: 14000

28000

2

Ponte a prueba 5

Cálculo de medias (variante funcional)

Vamos a resolver el caso práctico estudiado en el apartado de **Funciones** utilizando el **código funcional en lugar de bucles**. El objetivo es el mismo: crear una función que calcule la **media aritmética** de un conjunto de variable de valores pasados como parámetros.

Ejemplo de implementación

```
JS PonteAPrueba5.js X
Ponte a prueba > JS PonteAPrueba5.js > mediaAritmetica
1 // Autor: Mario Segura Abad
2 // Fecha: 24/10/2025
3
4 function mediaAritmetica(...numeros) {
5     //Si no hay números, devolvemos el valor 0
6     if(numeros.length === 0) {}
7         return 0;
8     }
9
10    // Sumamos todos los valores y dividimos entre la cantidad
11    return numeros.reduce((total, numero) => total + numero, 0) / numeros.length;
12 }
13
14 // Pruebas
15 console.log("Media 1:", mediaAritmetica(5, 10, 15));
16 console.log("Media 2:", mediaAritmetica(4, 8, 2));
17 console.log("Media 3:", mediaAritmetica(10, 15, 20));
18 console.log("Media 4:", mediaAritmetica(1, 2, 3));
19 console.log("Media 5:", mediaAritmetica(2, 10, 33));
```

Claves de resolución:

La función tiene que procesar un conjunto variable de parámetros, por lo que se deberán utilizar de nuevo los parámetros **rest**.

En cuanto a la elección de la función que se deberá utilizar, hay que tener en cuenta que queremos operar sobre un listado (*array*) y generar como resultado un **único valor**. ¿Cuál de las funciones estudiadas en este apartado está especializada en esa tarea?

DESARROLLO DE LA PRACTICA

Práctica DWEC – Unidad 2

Descripción

Este proyecto se centra en el uso de programación funcional en JavaScript para trabajar con arrays de objetos, aplicar transformaciones, filtrados, cálculos y recorridos complejos. Se han probado todos los scripts del apartado 5, documentado el código teórico y desarrollado el caso práctico extendido. El objetivo principal ha sido aplicar métodos como filter(), find(), map(), reduce() y forEach() para resolver problemas reales de forma declarativa, intentando evitar a toda costa bucles tradicionales y mejorando la legibilidad del código.

Contenido del archivo

Código del Apartado 5 de teoría

Se ha recopilado y probado todo el código mostrado en clase relacionado con el uso de funciones sobre arrays y objetos. Se ha trabajado con estructuras de datos complejas, como arrays de objetos, y se han aplicado métodos funcionales para extraer, transformar y presentar información. Cada archivo incluye comentarios con el nombre del autor y la fecha de realización. Se han capturado los resultados por consola como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – DWEC_U02_A05_TEO_1.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");

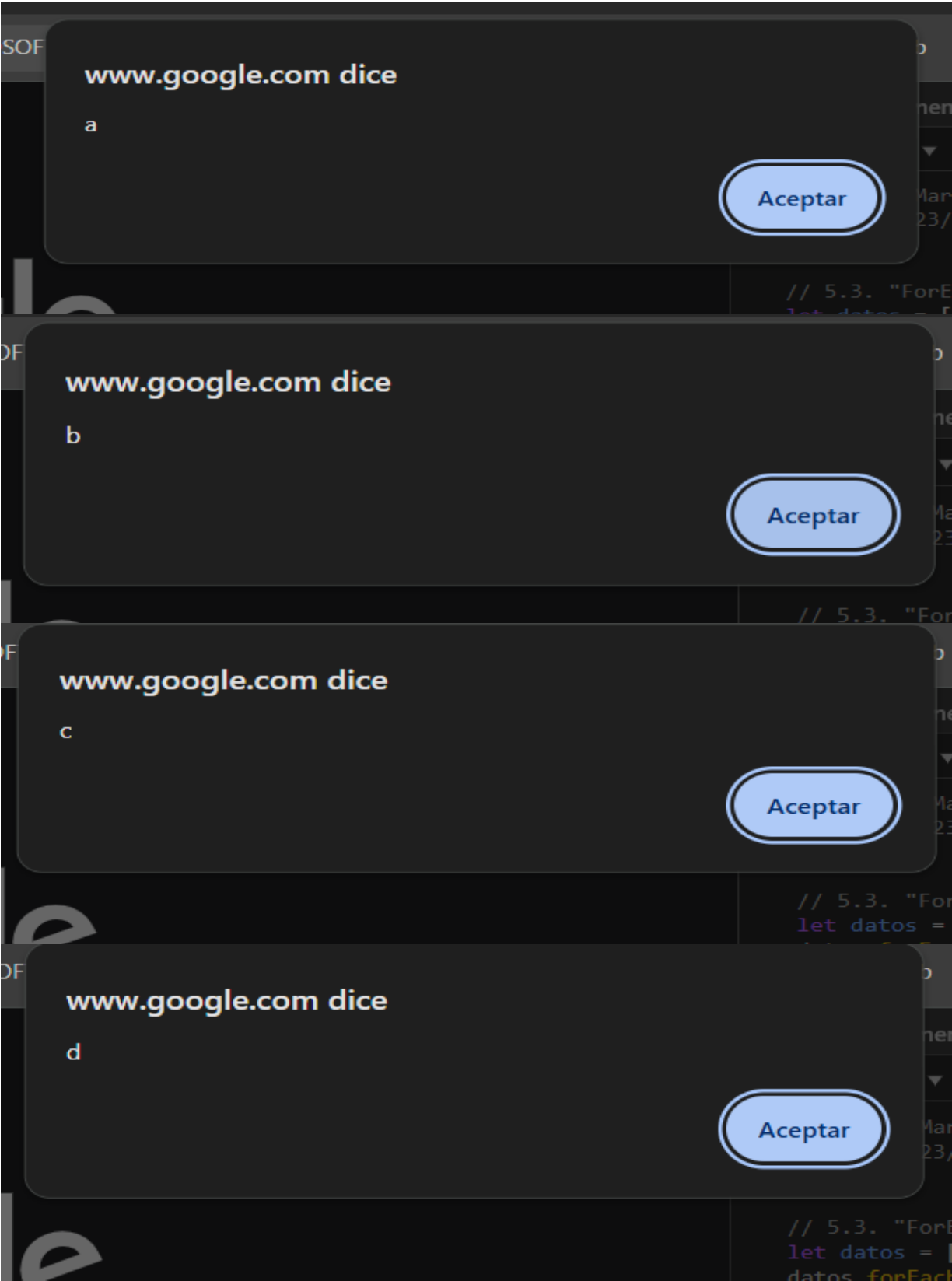
▶ (2) [14, 3] VM633:16
▶ (2) [14, 3] VM633:23
▶ (2) [14, 3] VM633:33
Autor: Mario Segura Abad VM633:36
Fecha de realización: 23/10/2025 VM633:37
← undefined
```

Salida por consola – DWEC_U02_A05_TEO_2.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");

▶ {nombre: 'Juan', apellido: 'Martínez', nif: 2} VM810:13
Autor: Mario Segura Abad VM810:16
Fecha de realización: 23/10/2025 VM810:17
← undefined
```

Salida por consola – DWEC_U02_A05_TEO_3.js



```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");

Procesando el elemento a, en la posición 0. El array es a,b,c,d
Procesando el elemento b, en la posición 1. El array es a,b,c,d
Procesando el elemento c, en la posición 2. El array es a,b,c,d
Procesando el elemento d, en la posición 3. El array es a,b,c,d
Autor: Mario Segura Abad
Fecha de realización: 23/10/2025
```

Salida por consola – DWEC_U02_A05_TEO_4.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");

(4) [1, 3, 5, 7]
(4) [2, 6, 10, 14]
Autor: Mario Segura Abad
Fecha de realización: 23/10/2025
```

Salida por consola – DWEC_U02_A05_TEO_5.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 24/10/2025");

11
Autor: Mario Segura Abad
Fecha de realización: 24/10/2025
```

Probar los Scripts del Apartado 5

Se han ejecutado los scripts adjuntos y se ha documentado su funcionamiento:
Ejemplo de ejecución:

Salida por consola – DWEC_U02_A05_01.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");

David Pérez
Juan Martínez
José Fernández
David Pérez
Juan Martínez
José Fernández
aa
Autor: Mario Segura Abad
Fecha de realización: 23/10/2025
```

Salida por consola – DWEC_U02_A03_02.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");
```

David Pérez	VM540:21
Juan Martínez	VM540:22
José Fernández	VM540:23
undefined	VM540:26
undefined	VM540:27
undefined	VM540:28
undefined	VM540:36
David	VM540:37
Autor: Mario Segura Abad	VM540:40
Fecha de realización: 23/10/2025	VM540:41

← undefined

Salida por consola – DWEC_U02_A05_03.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 23/10/2025");
```

▶ {rojo: {...}, verde: {...}, azul: {...}}	VM1226:27
2	VM1226:35
1	VM1226:37
1	VM1226:39
▶ (2) [{...}, {...}]	VM1226:42
Autor: Mario Segura Abad	VM1226:49
Fecha de realización: 23/10/2025	VM1226:50

← undefined

Versión 1: recorrido con `forEach` anidado y `Object.values()` para mostrar estructuras de objetos.

Ejemplo de ejecución:

```
=== Versión 1 ===
```

▶ {matricula: 'ABC1', color: 'rojo'}	VM1416:58
▶ {matricula: 'ABC4', color: 'rojo'}	VM1416:58
▶ {matricula: 'ABC2', color: 'verde'}	VM1416:58
▶ {matricula: 'ABC3', color: 'azul'}	VM1416:58
Autor: Mario Segura Abad	VM1416:64
Fecha de realización: 23/10/2025	VM1416:65

← undefined

Versión 2: presentación alternativa con formato un poco mas limpio y ordenado.
Ejemplo de ejecución:

```
=== Versión 2 ===
ABC1
rojo
ABC4
rojo
ABC2
verde
ABC3
azul
Autor: Mario Segura Abad
Fecha de realización: 23/10/2025
```

Versión 3: mejora visual usando funciones como reduce() y plantillas literales para generar salidas más legibles y estructuradas.
Ejemplo de ejecución:

```
=== Versión 3 ===
Matrícula: ABC1
Color: rojo
-----
Matrícula: ABC4
Color: rojo
-----
Matrícula: ABC2
Color: verde
-----
Matrícula: ABC3
Color: azul
-----
Autor: Mario Segura Abad
Fecha de realización: 23/10/2025
```

Caso Práctico 5 – Trabajo con arrays de objetos

1. Operaciones sobre array de coches

Se ha trabajado con un array de objetos que representa una colección de coches con propiedades como marca, modelo y precio. Mediante programación funcional se han realizado operaciones como:

- Filtrar elementos por marca o precio.
- Buscar el primer elemento que cumple una condición.
- Transformar el array añadiendo propiedades nuevas.
- Calcular valores agregados como el precio medio.

Estas operaciones han permitido aplicar los métodos filter(), find(), map() y reduce() de forma encadenada y eficiente.
Ejemplo de ejecución:

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 24/10/2025");

2DAW2      24/10/2025                                VM326:15
  ▶ (2) [{...}, {...}]                                VM326:16
  ▶ (3) [{...}, {...}, {...}]                          VM326:23
  ▶ {id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000} VM326:30
  ▶ (4) [{...}, {...}, {...}, {...}]                    VM326:40
  ▶ {total: 28000, cuenta: 2}                            VM326:54
                                                         VM326:55

2DAW2      24/10/2025                                VM326:61
Coches Seat:                                           VM326:64
  ▶ (2) [{...}, {...}]                                VM326:65
  ▶ {id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000} VM326:66
  ▶ {id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000} VM326:66
Coches Menores que 25000:                             VM326:70
  ▶ (3) [{...}, {...}, {...}]                          VM326:71
  ▶ {id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000} VM326:72
  ▶ {id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000} VM326:72
  ▶ {id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000} VM326:72
1º Coche Mayor que 20000:                             VM326:76
  ▶ {id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000} VM326:77
3                                                         VM326:78
Volkswagen                                             VM326:79
Golf                                                    VM326:80
21000                                                  VM326:81

Coches con Premium:                                   VM326:85
  ▶ (4) [{...}, {...}, {...}, {...}]                    VM326:86
  ▶ {id: 1, marca: 'Seat', modelo: 'Ibiza', precio: 10000, premium: false} VM326:87
  ▶ {id: 2, marca: 'Seat', modelo: 'Ateca', precio: 18000, premium: false} VM326:87
  ▶ {id: 3, marca: 'Volkswagen', modelo: 'Golf', precio: 21000, premium: true} VM326:87
  ▶ {id: 4, marca: 'Kia', modelo: 'Niro', precio: 30000, premium: true} VM326:87
Total y Cuenta Seat                                    VM326:91
  ▶ {total: 28000, cuenta: 2} ', precio medio:' 14000 VM326:92
28000                                                  VM326:93
2                                                         VM326:94
Autor: Mario Segura Abad                               VM326:97
Fecha de realización: 24/10/2025                       VM326:98
< undefined
```

Ponte a Prueba 5 - Cálculo de medias (variante funcional)

Se ha creado una función que calcula la media aritmética de un conjunto variable de valores utilizando parámetros rest y el método `reduce()`. Este ejercicio ha reforzado el uso de funciones reutilizables. Además, se ha comprendido cómo `reduce()` permite transformar un array en un único valor, lo que lo convierte en una herramienta clave para cálculos agregados.

Ejemplo de implementación:

```
JS PonteAPrueba5.js X
Ponte a prueba > JS PonteAPrueba5.js > mediaAritmetica
1 // Autor: Mario Segura Abad
2 // Fecha: 24/10/2025
3
4 function mediaAritmetica(...numeros) {
5     //Si no hay números, devolvemos el valor 0
6     if(numeros.length === 0) {
7         return 0;
8     }
9
10    // Sumamos todos los valores y dividimos entre la cantidad
11    return numeros.reduce((total, numero) => total + numero, 0) / numeros.length;
12 }
13
14 // Pruebas
15 console.log("Media 1:", mediaAritmetica(5, 10, 15));
16 console.log("Media 2:", mediaAritmetica(4, 8, 2));
17 console.log("Media 3:", mediaAritmetica(10, 15, 20));
18 console.log("Media 4:", mediaAritmetica(1, 2, 3));
19 console.log("Media 5:", mediaAritmetica(2, 10, 33));
```

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web: para ejecutar scripts con `console.log()` o integrarlos en una interfaz HTML (que no es mi caso).
- Node.js (opcional): para ejecutar los scripts directamente desde la terminal del equipo o del VS Code.

Pasos:

- Guardar todos los archivos `.js` en la carpeta raíz del proyecto.
- Abrir el o los archivos en el editor de código.
- Ejecutar el script en consola con Node.js o abrirlo directamente en la consola del navegador (F12).
- Ver los resultados por consola y comprobar que todas las funciones devuelven los valores esperados.

Objetivos de la práctica

- Aplicar programación funcional para recorrer, transformar y filtrar arrays de objetos.
- Utilizar métodos como `filter()`, `find()`, `map()`, `reduce()` y `forEach()` de forma eficiente.
- Crear funciones reutilizables con parámetros rest.
- Intentar mejorar la presentación de datos en consola.
- Documentar correctamente cada ejercicio con autoría y capturas.
- Presentar el proyecto en formato PDF junto con los archivos fuente y capturas de pantalla.

CONCLUSIÓN

Esta práctica nos ha permitido aplicar técnicas de programación funcional en JavaScript para resolver problemas reales de forma clara y eficiente. Se ha trabajado con arrays de objetos, aplicando transformaciones, filtrados y cálculos sin necesidad de bucles tradicionales, lo que mejora la legibilidad del código. Además, se ha reforzado el uso de funciones como `reduce()` para obtener resultados completamente agregados, y se ha mejorado la presentación de datos en consola. La práctica ha sido clave para entender cómo escribir código más limpio y reutilizable.